

	Imię i nazwisko: Aneta Kaczmarczyk	Rok 4 Semestr 1	Grupa 1
Ćwiczenia projektowe Zaawansowane Techniki Internetowe	Temat: Aplikacja internetowa do śledzenia przeczytanych i aktualnie czytanych książek (booktracking)		
Data wykonania: 01.06.26r.		Ocena:	

1. Cel projektu

Celem projektu było zaprojektowanie i implementacja nowoczesnej aplikacji internetowej typu booktracking, służącej do monitorowania własnej aktywności czytelniczej, planowania lektur oraz wchodzenia w interakcje ze społecznością innych czytelników.

Aplikacja miała na celu zintegrowanie lokalnej bazy danych (w której przechowywane są wrażliwe dane użytkowników, relacje społeczne oraz spersonalizowane dane czytelnicze) z zewnętrznym, globalnym API za pomocą zapytań GraphQL, umożliwiając pobieranie danych o tysiącach książek bez konieczności ich lokalnego składowania.

2. Użyte technologie

Backend: ASP.NET, Dapper, Npgsql, DbUp, BCrypt.Net, JWT, xUnit & Moq.

Frontend: Next.js, SWR, React Context API, Fetch Api.

Baza danych: PostgreSQL.

Narzędzia pomocnicze i integracje: Docker, GraphQL Api.

3. Zakładane funkcjonalności

W projekcie założono realizację następujących systemów funkcjonalnych:

- **System rejestracji i logowania (uwierzytelniania)** – Realizowanie dwuetapowego procesu zakładania konta, bezpieczne logowanie do indywidualnego profilu po podaniu adresu e-mail oraz hasła, z opcją zapamiętywania aktywnej sesji na

urządzeniu. Automatyczne sprawdzanie w locie, czy podany adres e-mail lub nazwa użytkownika nie są przypisane do innego konta. Zapewnianie opcji bezpiecznego wylogowywania się z aplikacji i unieważniania aktywnej sesji.

- **System wirtualnej biblioteki i monitorowania postępów (booktrackingu)** – Segregowanie posiadanych książek na wirtualnych półkach czytelnich (w trakcie czytania, przeczytane, planowane, porzucone) oraz precyzyjne śledzenie postępów czytelnich. Mierzenie czasu trwania sesji czytelnich w czasie rzeczywistym za pomocą wbudowanego stopera, ręczne raportowanie odbytych sesji czytania (podawanie stron, daty i czasu) oraz dodawanie i przeglądanie notatek powiązanych ze stronami książki. Definiowanie rocznego celu czytelniczego i śledzenie stopnia jego realizacji.
- **System wyszukiwania i rekomendacji książek** – Przeglądanie bazy książek, dynamiczne wyszukiwanie po tytule lub autorze, odczytywanie szczegółowych danych wybranej pozycji oraz filtrowanie wyników według gatunków i ich sortowanie. Generowanie personalizowanych rekomendacji nowych pozycji literackich na podstawie wybranego nastroju użytkownika, z automatycznym wykluczaniem książek znajdujących się już w bibliotece czytelnika.
- **System społecznościowy i interakcji ze znajomymi** – Wyszukiwanie profili innych czytelników, wysyłanie zaproszeń do grona znajomych, akceptowanie lub odrzucanie przychodzących zaproszeń oraz usuwanie relacji. Wyświetlanie dynamicznego strumienia aktywności pokazującego chronologiczne działania znajomych (rozpoczęcie czytania, ukończenie książki, dodanie oceny i recenzji).
- **System profilu i konfiguracji konta** – Personalizowanie danych profilu poprzez aktualizowanie nazwy wyświetlanej, biogramu opisowego oraz rocznego celu czytelniczego. Aktualizowanie hasła użytkownika wraz z analizowaniem jego siły na żywo przez wskaźnik trudności. Trwałe i bezpowrotne usuwanie konta z bazy danych wraz z powiązanymi relacjami, recenzjami i historią, po autoryzacji hasłem oraz ręcznym wpisaniu frazy potwierdzającej.

4. Schemat aplikacji

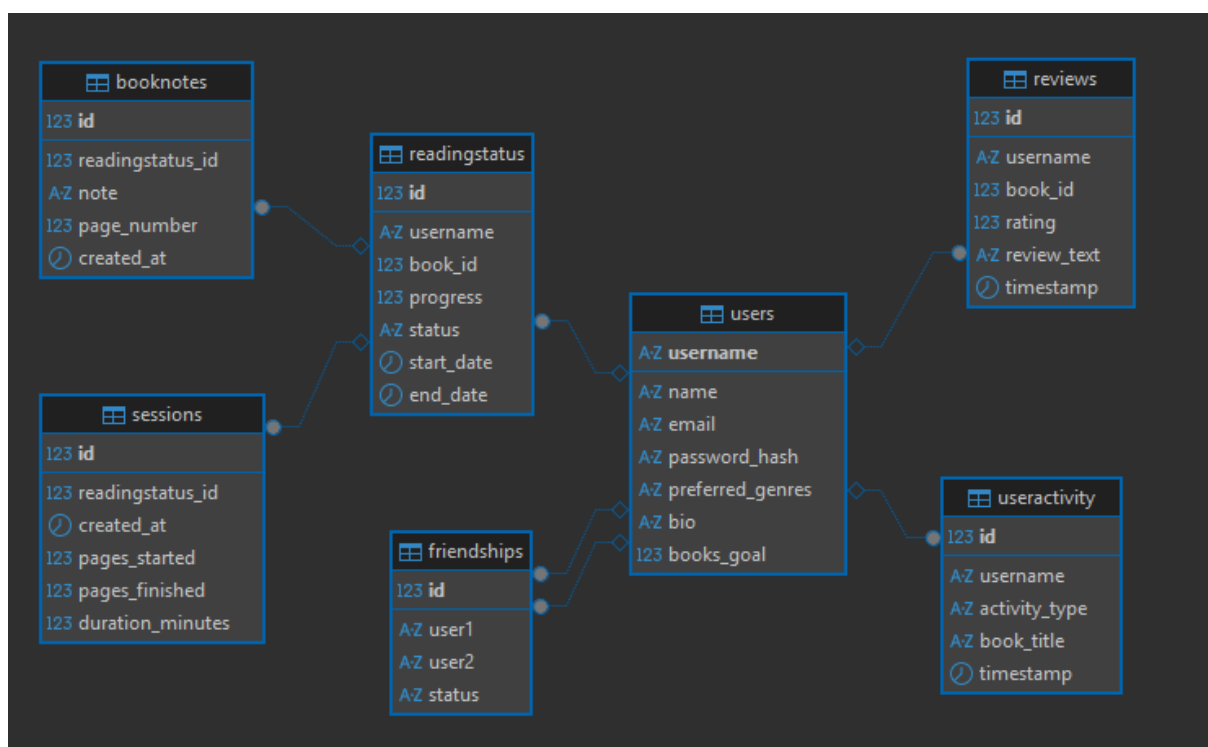
TrackMyBooks - app/

```
├─ backend/
│   └─ book-service/           # Kod źródłowy API w C# (ASP.NET Core)
│       ├── Controllers/       # Kontrolery API
│       ├── Repositories/      # Warstwa dostępu do bazy
│       ├── Models/           # Klasy encji i obiektów transferu danych (DT
│       ├── Program.cs         # Punkt startowy, konfiguracja JWT, CORS, DbUp, DI
│       └─ authorizationKey.txt # Klucz autoryzacji do zewnętrznego API Hardcover
├─ frontend/                  # Kod źródłowy aplikacji klienckiej (Next.js)
│   └─ src/
│       ├── app/              # Podstrony systemu (routing App Router)
│       └─ _context/          # AuthContext (zarządzanie sesją, cookies)
```

```
| | ├── _hooks/           # useToast Hook (powiadomienia)
| | |   └── _components/  # Navbar, Toast, statystyki i komponenty interfejsu
└── docker-compose.yml    # Konfiguracja wielokontenerowa środowiska Docker
└── README.md             # Dokumentacja projektu
```

5. Schemat bazy danych

Projekt bazy danych oparto na relacyjnym systemie PostgreSQL, w którym centralną encją jest tabela `users`, przechowująca profile oraz bezpieczne skróty haseł użytkowników. Z tabelą tą powiązana jest kluczowa tabela `readingstatus`, służąca do katalogowania książek na wirtualnych półkach oraz zapisywania aktualnego postępu lektury. Do każdego statusu książki przypisywane są relacyjne wpisy z tabel `booknotes` (osobiste notatki) oraz `sessions` (sesje mierzone stoperem), co pozwala na precyzyjne śledzenie historii czytelnictwa. Opinie i oceny czytelników są gromadzone w tabeli `reviews`. Relacje społeczne i wymianę informacji realizują tabele `friendships` (zaproszenia do znajomych) oraz `useractivity` (historia akcji zasilająca strumień aktywności). Cała struktura tabel jest automatycznie wdrażana i wersjonowana przy starcie serwera za pomocą biblioteki `DbUp`, co gwarantuje spójność schematu w środowisku `Docker`.



Rysunek 1. Schemat bazy danych.

6. Fragmenty kodu z omówieniem

a. System autoryzacji i zarządzania globalną sesją (AuthContext)

W celu zachowania tożsamości zalogowanego użytkownika pomiędzy odświeżeniami stron oraz unikania przechowywania wrażliwych tokenów JWT w magazynie LocalStorage, wdrożono scentralizowany kontener stanu autoryzacji (AuthProvider) po stronie klienckiej, współpracujący bezpośrednio z bezpiecznymi mechanizmami ciasteczek serwerowych.

```
const AuthContext = createContext<AuthContextType | undefined>(undefined);
// Provider zarządzający globalnym stanem uwierzytelnienia w całej
// aplikacji Next.js
export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);
  // Przy pierwszym załadowaniu strony automatycznie sprawdzamy, czy
  // użytkownik ma aktywną sesję
  useEffect(() => {
    fetchCurrentUser().finally(() => setLoading(false));
  }, []);
  const fetchCurrentUser = async () => {
    try {
      const res = await fetch("http://localhost:5000/api/user/me", {
        method: "GET",
        // credentials: "include" nakazuje przeglądarce dołączyć bezpieczne
        // ciasteczka (HttpOnly cookie z tokenem JWT)
        // do zapytania cross-origin (z localhost:3000 na backend na porcie
        // 5000)
        credentials: "include",
      });
      if (res.ok) {
        const data = await res.json();
        setUser(data);
      } else {
        setUser(null);
      }
    } catch (err) {
      setUser(null);
    }
  };
  // Umożliwia wymuszenie ponownego pobrania danych o użytkowniku
  const refreshUser = async () => {
    setLoading(true);
    await fetchCurrentUser();
    setLoading(false);
  };
}
```

```

};
const logout = async () => {
  try {
    await fetch("http://localhost:5000/api/user/logout", {
      method: "POST",
      credentials: "include", // Również wymagane, aby usunąć ciasteczko sesyjne
      // na porcie 5000
    }).then(() => setUser(null));
  } catch (err) {
    console.error("Logout failed:", err);
  } finally {
    setUser(null);
    // Przekierowanie użytkownika na stronę główną i pełne przeładowanie
    // w celu wyczyszczenia stanów aplikacji
    window.location.href = "/";
  }
};
return (
  <AuthContext.Provider value={{ user, loading, logout, refreshUser }}>
    {children}
  </AuthContext.Provider>
);
}
// Customowy hook ułatwiający dostęp do danych sesyjnych w każdym
// komponencie klienckim
// Zabezpieczenie na początku rzuca czytelny błąd, jeśli zapomnimy opakować
// aplikację w <AuthProvider>
export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) throw new Error("useAuth must be used within
  AuthProvider");
  return context;
};
};

```

Fragment kodu 2. Frontend. Globalny dostawca stanu sesji, który przy każdym starcie aplikacji lub odświeżeniu strony asynchronicznie odpytuje serwer o tożsamość użytkownika.

```

// Pobieranie danych aktualnie zalogowanego użytkownika na podstawie
// tożsamości zdekodowanej z tokena JWT
[Authorize]
[HttpGet("me")]
public async Task<IActionResult> GetCurrentUser()
{
  var userEmail = User.FindFirst(ClaimTypes.Email)?.Value;
  if (string.IsNullOrEmpty(userEmail))
  {
    return Unauthorized();
  }
  var user = await _userRepository.GetUserByEmail(userEmail);
  if (user == null)

```

```

    {
      return NotFound();
    }
    return Ok(user);
  }
};

```

Fragment kodu 2. Backend. Kod odpowiada za odczyt tożsamości zdekodowanego tokena JWT.

Prezentowane fragmenty kodu ilustrują mechanizm bezpiecznej synchronizacji sesji użytkownika. Warstwa kliencka (Fragment 1) inicjuje zapytanie asynchroniczne z parametrem *credentials*: "include", co wymusza na przeglądarce dołączenie do nagłówek żądania ciasteczka autoryzacyjnego, pomimo komunikacji cross-origin między różnymi portami lokalnymi. Po stronie serwerowej (Fragment 2) dekorator *[Authorize]* uruchamia wbudowane oprogramowanie pośredniczące do automatycznego przechwycenia, odszyfrowania i walidacji tokenu JWT. Po pomyślnej weryfikacji adres e-mail użytkownika jest bezpiecznie wyodrębniany ze zdekodowanych roszczeń (*User.FindFirst*), co pozwala na bezpieczne pobranie danych profilu z bazy danych.

b. System wyszukiwania i rekomendacji książek

Zaimplementowany moduł odpowiada za natychmiastowe wyszukiwanie książek po tytule lub autorze bez obciążania interfejsu (na frontendzie) oraz za generowanie spersonalizowanych rekomendacji na podstawie nastroju i historii biblioteki zalogowanego użytkownika (na backendzie).

```

// Wykorzystanie useMemo zapobiega ponownemu filtrowaniu i sortowaniu tysięcy
książek przy każdym renderze.
// Zapobiega to opóźnieniu (input lag) podczas wpisywania tekstu w
wyszukiwarke.
const filtered = useMemo(() => {
  return allBooks
    .filter(b => genre === "All" || b.cached_Tags.Genre?.some((g: GenreTag) =>
g.tag === genre))
    .filter(b => !query || b.title.toLowerCase().includes(query.toLowerCase())
||
b.contributions[0]?.author?.name.toLowerCase().includes(query.toLowerCase()))
    .sort((a, b) => {
      if (sort === "Rating: highest") return b.rating - a.rating;
      if (sort === "Title: A-Z") return a.title.localeCompare(b.title);
      if (sort === "Newest first") return (b.release_Date || "0000-00-
00").localeCompare(a.release_Date || "0000-00-00");
      if (sort === "Oldest first") return (a.release_Date || "9999-99-
99").localeCompare(b.release_Date || "9999-99-99");
      return 0;
    });
});

```

```
}, [allBooks, genre, sort, query]); // Efekt przelicza się tylko przy zmianie tych zależności
```

Fragment kodu 3. Frontend. Wydajne filtrowanie, wyszukiwanie i sortowanie lokalnego katalogu książek z wykorzystaniem useMemo w celu zapobiegania opóźnieniom interfejsu.

```
[HttpGet("getRecommendations")]
public async Task<IActionResult> GetRecommendations([FromQuery] string
mood, [FromQuery] string? username)
{
    // 1. Mapowanie wybranego "nastroju" na konkretne tagi i gatunki
literackie, z których korzysta API
    var keywords = mood.ToLower() switch
    {
        "relax" => new List<string> { "romance", "humor", "comedy",
"strength" },
        "adventure" => new List<string> { "fantasy", "sci-fi", "science
fiction", "adventure" },
        "mind" => new List<string> { "science", "philosophy", "nonfiction"
},
        "emotion" => new List<string> { "drama", "biography", "memoir" },
        "mystery" => new List<string> { "thriller", "mystery", "crime",
"horror" },
        "wonder" => new List<string> { "magical realism", "high fantasy"
},
        _ => new List<string> { "fiction" }
    };

    // Pobieranie szerszej puli (40 najlepszych książek), żeby mieć z
czego filtrować
    var allBooks = await _client.GetRecommendations(limit: 40);

    // 2. Identyfikacja użytkownika - sprawdzamy, czy request pochodzi od
zalogowanego usera
    string? activeUsername = null;

    if (!string.IsNullOrEmpty(username) && username.Trim().ToLower()
!= "null" && username.Trim().ToLower() != "undefined")
    {
        activeUsername = username.Trim();
    }
    else if (User.Identity != null && User.Identity.IsAuthenticated &&
!string.IsNullOrEmpty(User.Identity.Name))
    {
        var tokenName = User.Identity.Name.Trim();
        if (tokenName.ToLower() != "null" && tokenName.ToLower() !=
"undefined")
        {
            activeUsername = tokenName;
        }
    }
}
```

```

    }

    var userOwnedBookIds = new List<int>();

    // 3. Zabezpieczenie przed polecaniem książek, które użytkownik już
przeczytał lub ma na liście
    if (!string.IsNullOrEmpty(activeUsername))
    {
        var readingStatuses = await
_booksdbContext.GetUserReadingStatus(activeUsername);

        userOwnedBookIds = readingStatuses
            .Select(rs => rs.Book_Id)
            .ToList();
    }
    // 4. Zaawansowane filtrowanie i transformacja wyników
var response = allBooks
    .Where(b => b.Id.HasValue)
    .Where(b => !userOwnedBookIds.Contains(b.Id.Value))
    .Select(b =>
    {
        // Sprawdzamy, czy tagi książki pokrywają się ze słowami
kluczowymi wybranego nastroju
        var matchedTag = b.Cached_Tags?
            .SelectMany(pair => pair.Value)
            .FirstOrDefault(t => t != null &&
!string.IsNullOrEmpty(t.Tag) && keywords.Contains(t.Tag.ToLower()));
        string primaryGenre = matchedTag != null
            ? matchedTag.Tag
            :
(b.Cached_Tags?.FirstOrDefault().Value?.FirstOrDefault()?.Tag ??
keywords.FirstOrDefault()) ?? "Fiction");

        return new { Book = b, PrimaryGenre = primaryGenre, IsMatch =
matchedTag != null };
    })
    .OrderByDescending(x => x.IsMatch) // Najlepsze dopasowania lądują
na początku listy
    .Take(10) // Ograniczamy ostateczny wynik do 10 książek
    .Select(x =>
    {
        // Generowanie dynamicznego opisu
        string reasonText = string.IsNullOrEmpty(activeUsername)
            ? $"\"If you love the vibe of {x.PrimaryGenre} - this
highly-rated pick from the {mood} section is an absolute must-read!\""
            : $"\"Based on your interest in {x.PrimaryGenre} - you are
looking for a story with a deeper atmosphere and tension.\"";
        // Mapowanie na obiekt DTO wysyłany do klienta
        return new RecommendationDto

```

```

        {
            Book_Id = x.Book.Id.Value,
            Title = x.Book.Title,
            AuthorName =
x.Book.Contributions?.FirstOrDefault()?.Author?.Name ?? "Unknown Author",
            ImageUrl = x.Book.Cached_Image?.Url ?? "",
            Rating = x.Book.Rating,
            PrimaryGenre = x.PrimaryGenre.ToUpper(),
            Reason = reasonText
        };
    }).ToList();

    return Ok(response);
}

```

Fragment kodu 4. Backend. Pobieranie rekomendacji z zewnętrznego API na podstawie nastroju wraz z filtrowaniem i odrzucaniem książek posiadanych już przez użytkownika w bazie.

System ten łączy optymalizację renderowania na kliencie z zaawansowanym filtrowaniem po stronie serwera. Warstwa kliencka (Fragment kodu 3) wykorzystuje hook `useMemo` do zapamiętywania przefiltrowanej i posortowanej listy książek. Dzięki temu, gdy użytkownik wpisuje znaki w polu wyszukiwania, React nie przelicza całej bazy na nowo przy każdym uderzeniu w klawisz, co całkowicie eliminuje opóźnienia interfejsu. Po stronie serwerowej (Fragment kodu 4), silnik rekomendacji odpytuje zewnętrzne API Hardcover, a następnie pobiera bibliotekę zalogowanego użytkownika (`GetUserReadingStatus`) z PostgreSQL. Filtrując wyniki, automatycznie odrzuca książki, które użytkownik ma już na swoich półkach, i mapuje wynik na zoptymalizowany pod kątem transferu obiekt `RecommendationDto`.

c. System społecznościowy i interakcji ze znajomymi

Moduł odpowiada za agregowanie danych o sieci relacji społecznościowych. Kluczowym wyzwaniem było wydajne pobranie aktywności znajomych po stronie bazy (backend) oraz ułożenie ich w spójną, jednolitą oś czasu (frontend).

```

<div className="friends-activity">
  {friendsData
    .filter(f => f.friendshipStatus === "accepted")
    // flatMap łączy listy pojedynczych tablic aktywności znajomych w
    jedną, płaską tablicę nadrzędną
    .flatMap(f => {
      return f.activities.map(a => ({
        ...a,
        friendName: f.name || f.username,
        friendUsername: f.username
      }));
    })
    // Sortowanie całej osi czasu od najświeższych wpisów (chronologia)
  }
}

```

```

        .sort((b, c) => new Date(c.timestamp).getTime() - new
Date(b.timestamp).getTime())
        .slice(0, 10) // Wyświetlamy maksymalnie 10 ostatnich wydarzeń
        .map((a, i) => (
            <div className="fbc-activity-row" key={i}>
                <div className="fbc-activity-avatar">
                    {a.friendName.split(" ").map(n =>
n.charAt(0).toUpperCase()).join("").slice(0, 2)}
                </div>

                <div className="fbc-activity-text">
                    <span className="fbc-activity-name">{a.friendUsername}</span>
                    {" "} <span className="fbc-activity-
action">{a.activityType}</span>{" "}
                    <em className="fbc-activity-book">„{a.bookTitle}“</em>
                </div>

                    <span className="activity-
time">{formatTimeAgo(a.timestamp)}</span>
                </div>
            )
        )
    </div>

```

Fragment kodu 5. Frontend. Płaskie mapowanie (flatMap) oraz sortowanie chronologiczne aktywności zaakceptowanych znajomych w celu wygenerowania jednolitego strumienia wydarzeń.

```

public async Task<IEnumerable<FriendWithBooksDto>> GetFriendsData(string
username)
{
    using var connection = _db.CreateConnection();

    // Pobieramy relacje znajomości wraz ze stanem czytelnictwa znajomych
    var friendsQuery = @"
        SELECT
            f.status AS FriendshipStatus,
            u.username AS FriendUsername,
            u.name AS FriendName,
            CASE WHEN f.user1 = @Username THEN true ELSE false END AS
IsInitiator,
            rs.id AS StatusId,
            rs.book_id AS BookId,
            rs.status AS ReadingStatus,
            rs.progress AS Progress,
            rs.start_date AS StartDate,
            rs.end_date AS EndDate
        FROM Friendships f
        JOIN Users u ON (f.user1 = u.username OR f.user2 = u.username)
        LEFT JOIN ReadingStatus rs ON u.username = rs.username
        WHERE (f.user1 = @Username OR f.user2 = @Username)
            AND u.username != @Username";

```

```

// Pobieramy historie aktywności wszystkich znajomych w jednym zapytaniu
var activityQuery = @"
    SELECT
        ua.username AS Username,
        ua.activity_type AS ActivityType,
        ua.book_title AS BookTitle,
        ua.timestamp AS Timestamp
    FROM Friendships f
    JOIN Users u ON (f.user1 = u.username OR f.user2 = u.username)
    JOIN UserActivity ua ON u.username = ua.username
    WHERE (f.user1 = @Username OR f.user2 = @Username)
        AND u.username != @Username";

// Pobieramy recenzje wszystkich znajomych w jednym zapytaniu
var reviewQuery = @"
    SELECT
        r.username AS Username,
        r.id AS Id,
        r.book_id AS BookId,
        r.rating AS Rating,
        r.review_text AS ReviewText,
        '' AS BookTitle,
        r.timestamp AS Timestamp
    FROM Friendships f
    JOIN Users u ON (f.user1 = u.username OR f.user2 = u.username)
    JOIN Reviews r ON u.username = r.username
    WHERE (f.user1 = @Username OR f.user2 = @Username)
        AND u.username != @Username";

// Asynchroniczne wywołanie wszystkich trzech zapytań SQL
var friendRows = await
connection.QueryAsync<RawFriendshipRow>(friendsQuery, new { Username =
username });
var activityRows = await
connection.QueryAsync<RawActivityRow>(activityQuery, new { Username = username
});
var reviewRows = await connection.QueryAsync<RawReviewRow>(reviewQuery,
new { Username = username });

// Tworzenie słowników przyspieszających wyszukiwanie powiązanych danych w
pamięci
var activitiesLookup = activityRows
    .GroupBy(a => a.Username)
    .ToDictionary(g => g.Key, g => g.ToList());

var reviewsLookup = reviewRows
    .GroupBy(r => r.Username)
    .ToDictionary(g => g.Key, g => g.ToList());

```

```

// Składanie końcowego obiektu DTO
var result = friendRows
    .GroupBy(r => new { r.FriendUsername, r.FriendName,
r.FriendshipStatus, r.IsInitiator }) // Grupowanie po znajomym
    .Select(g => new FriendWithBooksDto
    {
        Username = g.Key.FriendUsername,
        Name = g.Key.FriendName,
        FriendshipStatus = g.Key.FriendshipStatus,
        IsInitiator = g.Key.IsInitiator,

        // Mapowanie statusów książek, odfiltrowując puste wiersze
        powstałe przy LEFT JOIN
        ReadingStatuses = g
            .Where(r => r.StatusId != null)
            .Select(r => new FriendReadingStatusDto
            {
                Id = r.StatusId!.Value,
                Book_Id = r.BookId!.Value,
                Status = r.ReadingStatus ?? string.Empty,
                Progress = r.Progress ?? 0,
                Start_Date = r.StartDate,
                End_Date = r.EndDate
            }).ToList(),

        // Pobieranie aktywności ze słownika pamięci podręcznej
        Activities = activitiesLookup.ContainsKey(g.Key.FriendUsername)
            ? activitiesLookup[g.Key.FriendUsername].Select(a => new
FriendActivityDto
            {
                ActivityType = a.ActivityType,
                BookTitle = a.BookTitle,
                Timestamp = a.Timestamp
            }).ToList()
        : new List<FriendActivityDto>(),

        // Pobieranie recenzji ze słownika pamięci podręcznej
        Reviews = reviewsLookup.ContainsKey(g.Key.FriendUsername)
            ? reviewsLookup[g.Key.FriendUsername].Select(r => new
FriendReviewDto
            {
                Id = r.Id,
                Book_Id = r.BookId,
                Rating = r.Rating,
                Review_Text = r.ReviewText,
                Book_Title = r.BookTitle,
                Timestamp = r.Timestamp
            }).ToList()
    }

```

```

        : new List<FriendReviewDto>()
    })
    .ToList();
    return result;
}

```

Fragment kodu 6. Backend. Zaawansowana, zoptymalizowana agregacja bazy danych pobierająca relacje, aktywności oraz opinie znajomych za pomocą osobnych, zintegrowanych zapytań SQL.

Moduł ten rozwiązuje częsty problem wydajnościowy w systemach społecznościowych. Pobieranie listy aktywności każdego znajomego w osobnej pętli doprowadziłoby do przeciążenia bazy danych. Po stronie serwerowej (Fragment kodu 6) zastosowano optymalizację: pobierane są zbiorcze wyniki za pomocą wydajnych zapytań złączeniowych SQL, a ich mapowanie odbywa się w pamięci RAM serwera przy użyciu szybkiego grupowania `Dapper` i słowników `activitiesLookup`. Po stronie klienckiej (Fragment kodu 5), frontend filtruje tylko zaakceptowane relacje, a za pomocą metody `flatMap` spłaszcza odrębne tablice aktywności od różnych znajomych do jednej wspólnej listy, którą następnie sortuje chronologicznie, tworząc spójny strumień wydarzeń dla użytkownika.

d. System wirtualnej biblioteki i monitorowania postępów

Moduł odpowiada za monitorowanie czasu czytania oraz zarządzanie wirtualnymi półkami bibliotecznymi. Kluczowym wyzwaniem było bezkonfliktowe mierzenie sekund sesji na kliencie oraz precyzyjna, dwukierunkowa aktualizacja postępu i statusu książki w bazie danych.

```

    // Referencja useRef zapobiega ponownemu renderowaniu strony przy zmianie
    // sekund stopera
    const intervalRef = useRef<ReturnType<typeof setInterval> | null>(null);
    const [timerActive, setTimerActive] = useState(false);
    const [timerSeconds, setTimerSeconds] = useState(0);

    // Kontrola interwału stopera czytania co 1 sekundę (1000 ms)
    useEffect(() => {
        if (timerActive) {
            intervalRef.current = setInterval(() => setTimerSeconds(s => s + 1),
1000);
        } else {
            if (intervalRef.current) clearInterval(intervalRef.current);
        }
        // Funkcja czyszcząca (cleanup) usuwa timer z przeglądarki przy odmontowaniu
        // strony
        return () => { if (intervalRef.current) clearInterval(intervalRef.current);
        };
    }, [timerActive]);

    const finishSession = async () => {
        setTimerActive(false);
    }

```

```

    const endP = Math.min(bookData?.book.pages || 1, Math.max(currentPage,
+sessionEndPage || currentPage));
    const minutes = Math.round(timerSeconds / 60);

    if (endP - sessionStartPage > 0 || minutes > 0) {
        // Wywołanie API w celu zapisania sesji (czasu i stron) w bazie danych
        PostgreSQL
        await fetch(`http://localhost:5000/api/books/createSession`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            credentials: "include",
            body: JSON.stringify({ readingStatus_Id: readingStatus?.id,
pages_Started: sessionStartPage, pages_Finished: endP, duration_Minutes:
minutes, log_Date: new Date().toISOString().split("T")[0] })
        });

        // Wywołanie API w celu aktualizacji nadrzędnego postępu książki
        await fetch(`http://localhost:5000/api/books/updateProgress`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            credentials: "include",
            body: JSON.stringify({ book_Id: slug, username: user?.username,
progress: endP, isFinished: endP >= (bookData?.book.pages || 1) })
        });
        refreshUser?.(); // Odświeżenie danych globalnych kontekstu
    }
};

```

Fragment kodu 7. Frontend. Stoper czasu rzeczywistego wbudowany w panel czytania, odliczający sekundysesji z wykorzystaniem referencji w celu uniknięcia ponownych renderów.

```

// Zapisuje odbytą sesję czytania w tabeli Sessions
public async Task CreateSession(int readingStatusId, int pagesStarted, int
pagesFinished, int durationMinutes, DateTime logDate)
{
    using var connection = _db.CreateConnection();
    var query = "INSERT INTO Sessions (readingStatus_id, pages_started,
pages_finished, duration_minutes, created_at) VALUES (@ReadingId,
@PagesStarted, @PagesFinished, @DurationMinutes, @LogDate)";
    await connection.ExecuteAsync(query, new { ReadingId = readingStatusId,
PagesStarted = pagesStarted, PagesFinished = pagesFinished, DurationMinutes =
durationMinutes, LogDate = logDate });
}

// Aktualizuje ogólny postęp i status książki w tabeli ReadingStatus
public async Task UpdateProgress(string username, int bookId, int progress,
bool isFinished)
{
    using var connection = _db.CreateConnection();
    if (isFinished)

```

```

{
    // Jeśli książka skończona, automatycznie ustawiamy status na 'read'
    (przeczytane) i datę końcową
    var query = "UPDATE ReadingStatus SET progress = @Progress, end_date =
CURRENT_DATE, status = 'read' WHERE username = @Username AND book_id =
@BookId";
    await connection.ExecuteAsync(query, new { Username = username, BookId
= bookId, Progress = progress });
}
else
{
    // Jeśli w trakcie czytania, dbamy o to, by status wynosił 'reading'
    var query = "UPDATE ReadingStatus SET progress = @Progress, status =
'reading', end_date = NULL WHERE username = @Username AND book_id = @BookId";
    await connection.ExecuteAsync(query, new { Username = username, BookId
= bookId, Progress = progress });
}
}

```

Fragment kodu 8. Backend. Metody zapisu nowej sesji oraz asynchronicznej aktualizacji ogólnego postępu i statusu książki w bazie danych PostgreSQL.

Mechanizm ten realizuje asynchroniczne śledzenie postępów czytelnictwa. Warstwa kliencka (Fragment kodu 7) wykorzystuje referencję `intervalRef` do bezkonfliktowego odmierzenia czasu sesji, co pozwala na zachowanie tożsamości timera między kolejnymi cyklami renderowania i zapobiega niekontrolowanemu przyspieszaniu stopera. Po zatrzymaniu stopera, funkcja `finishSession` przelicza sekundy na minuty i wysyła asynchroniczne żądania POST do serwera. Warstwa serwerowa (Fragment kodu 8) odbiera te dane i za pomocą Dappera wykonuje dwa zoptymalizowane zapytania modyfikujące bazę PostgreSQL: rejestruje rekord sesji w tabeli `Sessions` oraz aktualizuje postęp i warunkowo status lektury w tabeli `ReadingStatus`.

e. System profilu i konfiguracji konta

Moduł odpowiada za konfigurację danych profilowych oraz wrażliwe operacje bezpieczeństwa konta. Kluczowym wyzwaniem było stworzenie reaktywnego kalkulatora siły hasła na kliencie oraz podwójnie zabezpieczonej procedury usuwania profilu na serwerze.

```

// Wyrażenia regularne badają obecność wielkich liter, cyfr i znaków
specjalnych w locie (przy każdej zmianie stanu)
const strength = (() => {
    if (!password) return 0;
    let s = 1;
    if (/[A-Z]/.test(password)) s++;           // Sprawdzenie obecności wielkiej
litery
    if (/[0-9]/.test(password)) s++;           // Sprawdzenie obecności cyfry
    if (/^[A-Za-z0-9]/.test(password)) s++;    // Sprawdzenie obecności znaku
specjalnego

```

```

    if (password.length >= 8) s++; // Sprawdzenie bezpiecznej
    długości
    else s = 0; // Jeśli hasło ma poniżej 8 znaków, automatycznie otrzymuje
    najniższy stopień (0)
    return s;
  })();

const strengthLabel = ["", "Weak", "Medium", "Strong", "Very
Strong"][strength];
const strengthColor = ["", "#e05252", "#e09452", "#c9a84c",
"#52b788"][strength];

```

Fragment kodu 9. Frontend. Analizator siły nowego hasła obliczany w czasie rzeczywistym z wykorzystaniem wyrażeń regularnych w celu wymuszenia silnej ochrony konta.

```

[HttpPost("delete")]
public async Task<IActionResult> DeleteAccount([FromBody]
DeleteAccountRequest request)
{
    // 1. Pobranie danych zalogowanego użytkownika
    var user = await _userRepository.GetUserByEmail(request.Email);

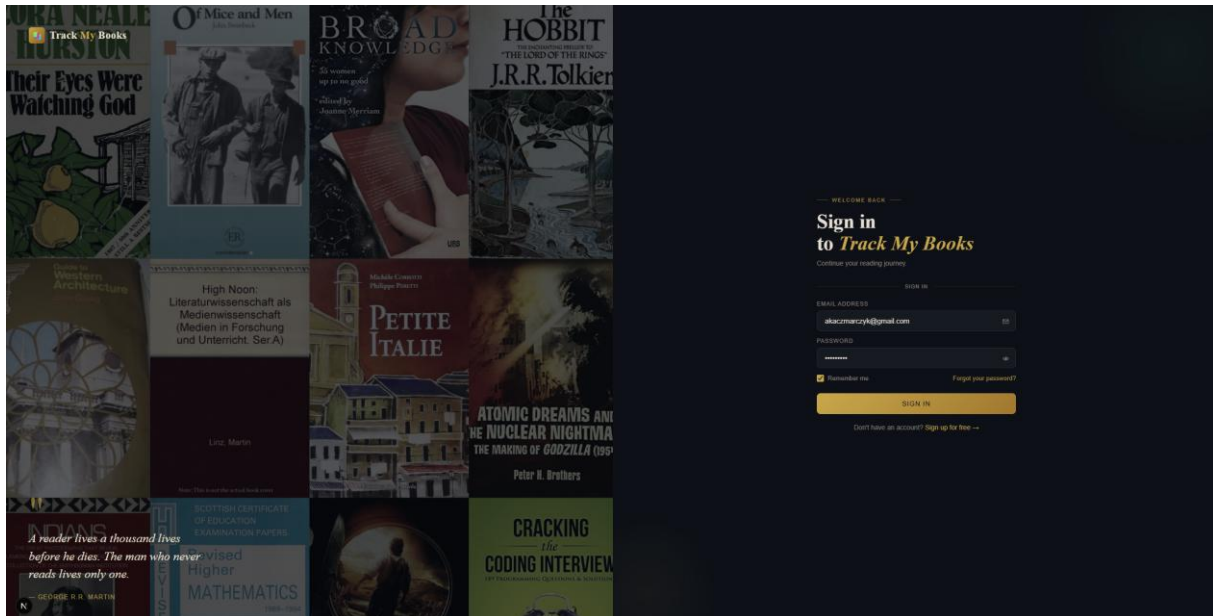
    // 2. Podwójna weryfikacja intencji usunięcia konta:
    // a) sprawdzenie poprawności hasła BCryptem,
    // b) zweryfikowanie poprawnego, ręcznego wpisania frazy blokującej
    "DELETE ACCOUNT"
    if (user == null || !BCrypt.Net.BCrypt.Verify(request.Password,
user.Password_Hash) || request.Confirmation != "DELETE ACCOUNT")
    {
        Console.WriteLine($"Delete account failed: user={user},
confirmationValid={request.Confirmation == "DELETE ACCOUNT"}");
        return Unauthorized("Invalid password");
    }

    // 3. Wywołanie repozytorium w celu bezpowrotnego skasowania
    wszystkich danych transakcyjnych użytkownika
    await _userRepository.DeleteUser(request);
    // 4. Automatyczne unieważnienie sesji (usunięcie ciasteczka JWT z
    przeglądarki)
    Logout();
    return Ok("Account deleted!");
}

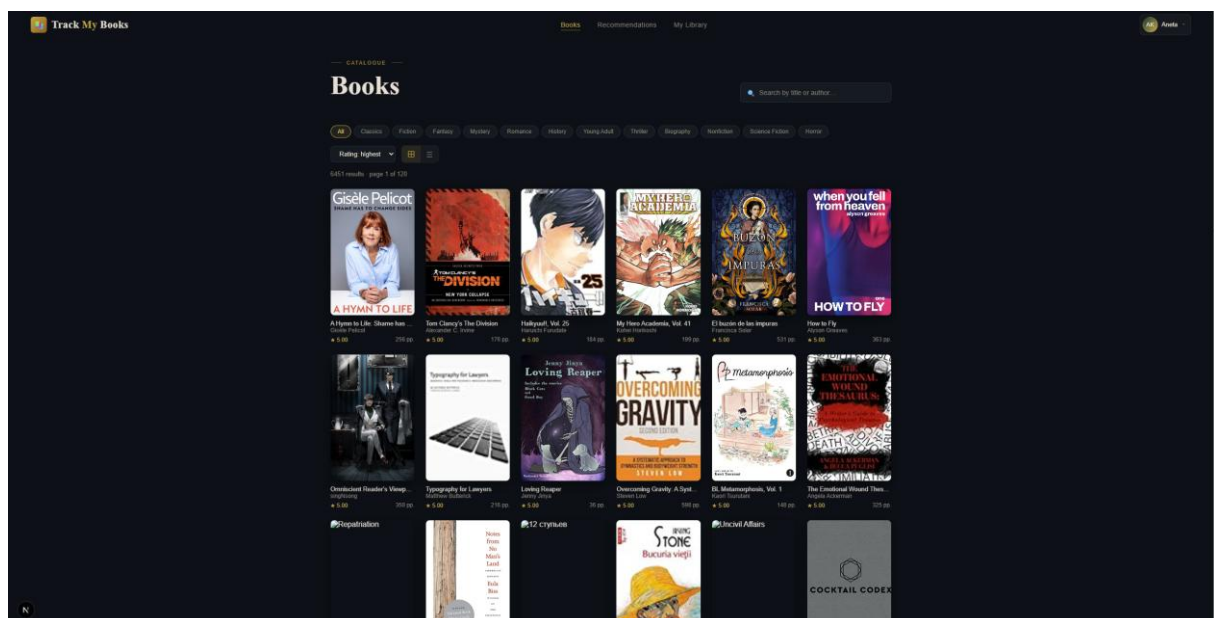
```

Fragment kodu 10. Backend. Metoda niszczenia konta użytkownika usuwająca wszelkie powiązane relacje, opinie i wpisy aktywności z bazy danych po podwójnej weryfikacji.

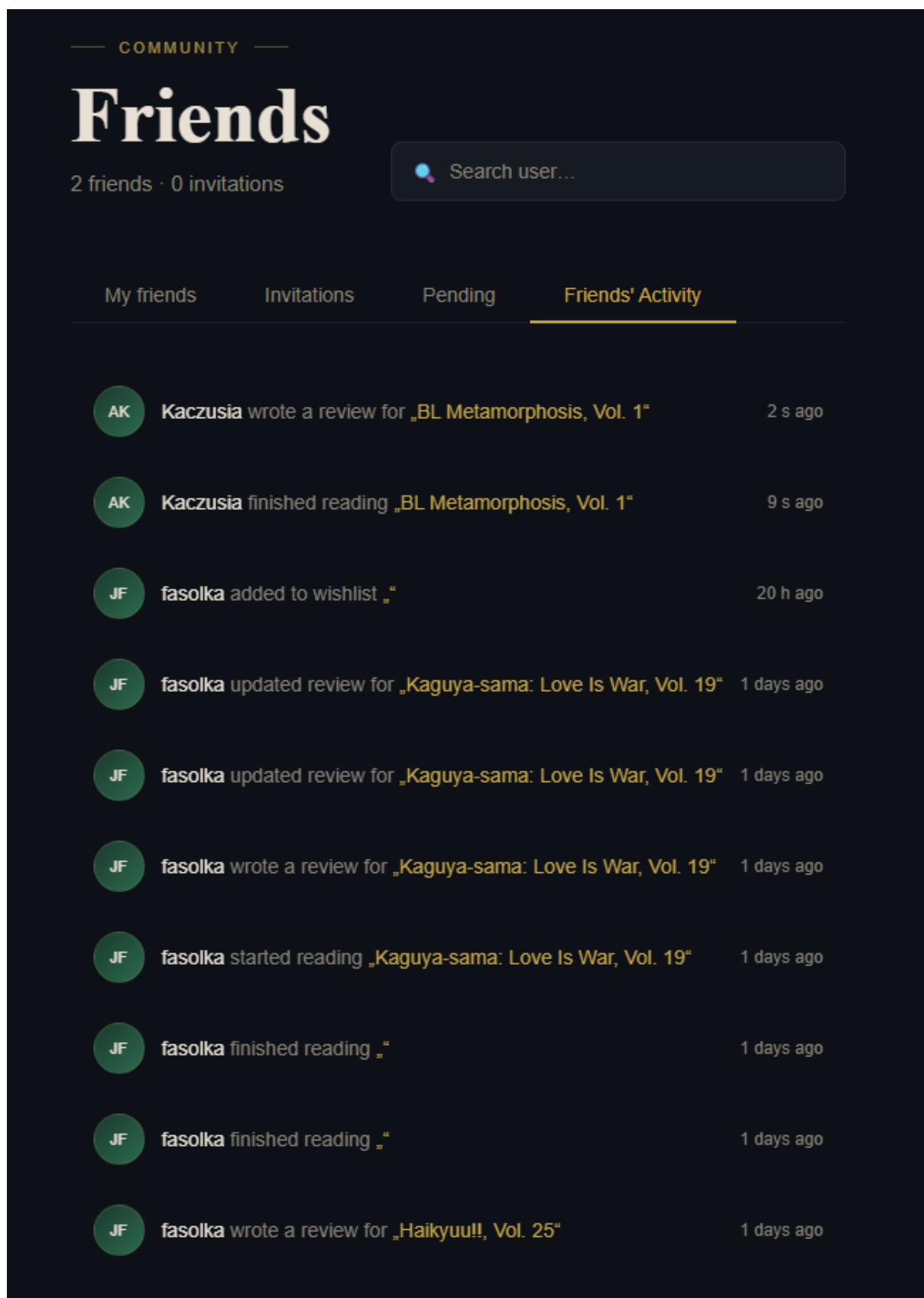
Zaimplementowany moduł łączy walidację na kliencie z restrykcyjnymi mechanizmami bezpieczeństwa na serwerze. Warstwa kliencka (Fragment kodu 9) za pomocą reaktywnego kodu w locie ocenia siłę hasła przy użyciu wyrażeń regularnych (Regex) i blokuje możliwość wystąpienia żądania, jeśli hasło nie spełnia minimalnych wymogów bezpieczeństwa. Z kolei warstwa serwerowa (Fragment kodu 10) przy operacji usuwania konta wdraża procedurę dwustopniowej weryfikacji. Przed wywołaniem asynchronicznej



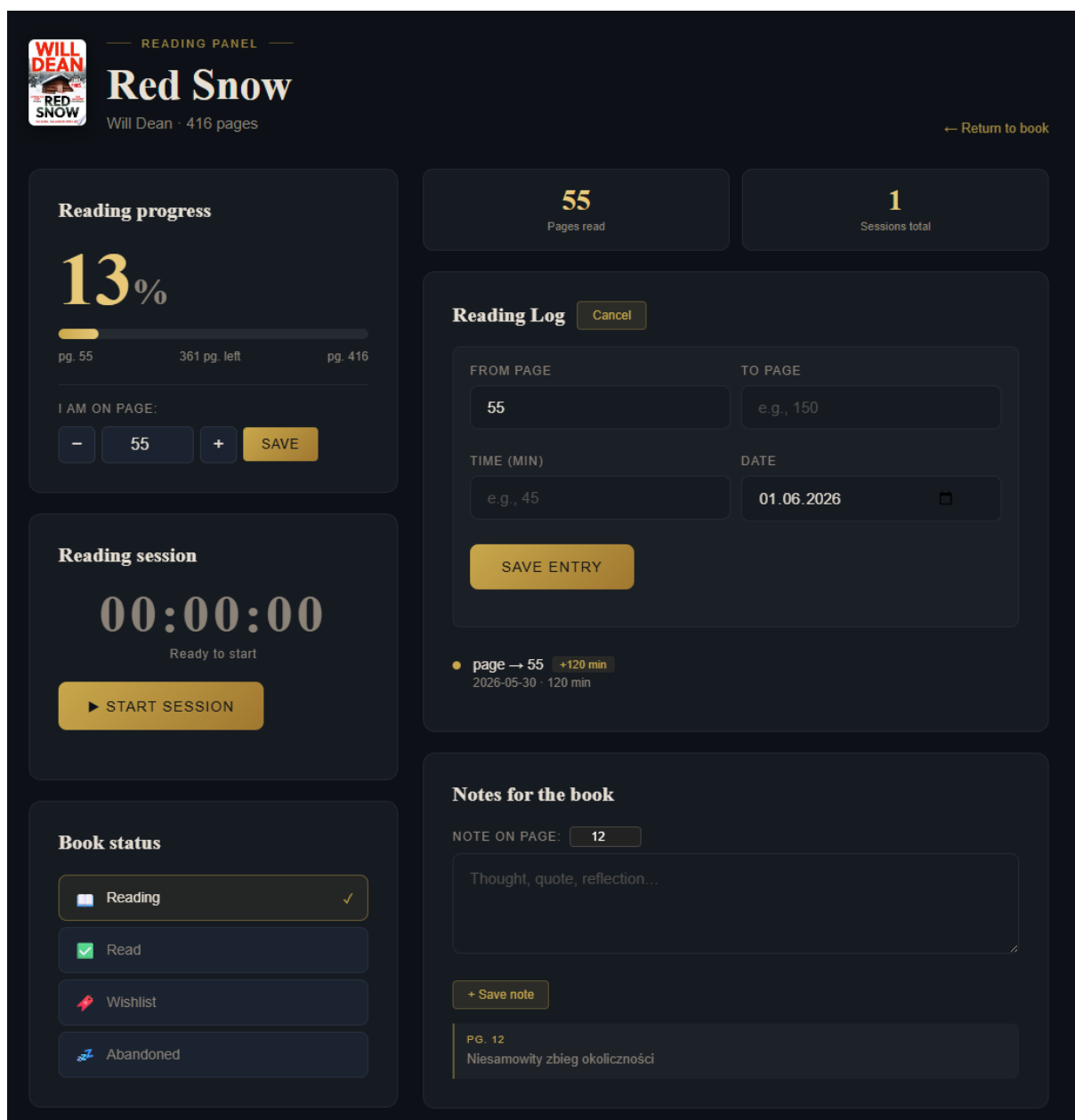
Rysunek 2. Panel logowania i autoryzacji sesji użytkownika – ekran powiązany z Fragmentem kodu 1 oraz Fragmentem kodu 2.



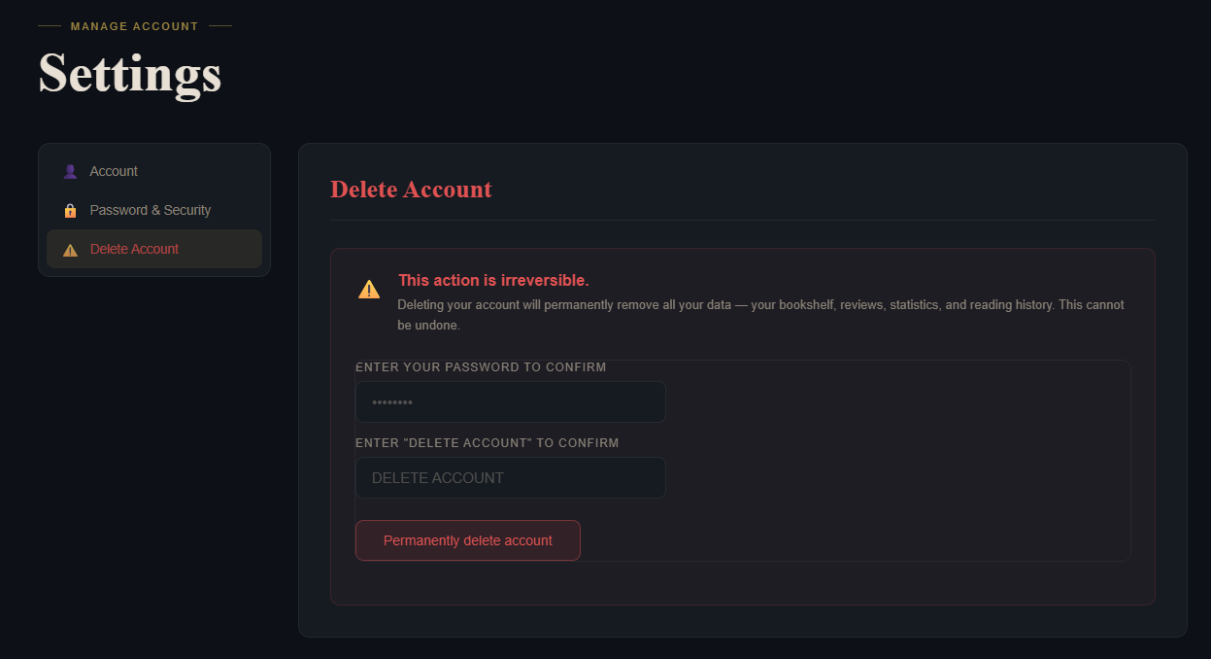
Rysunek 3. Katalog książek z dynamicznym filtrowaniem i wyszukiwaniem - ekran powiązany z Fragmentem kodu 3 oraz Fragmentem kodu 4.



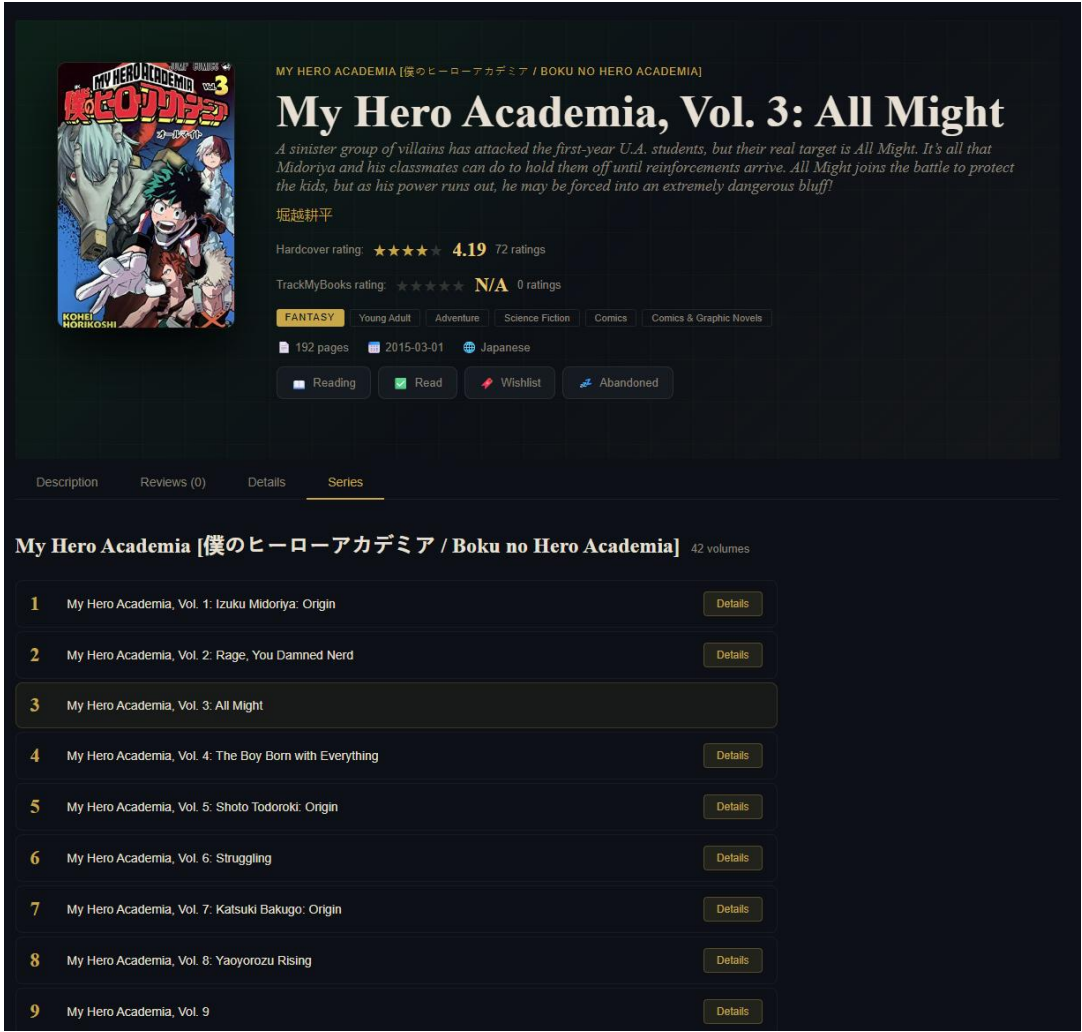
Rysunek 4. Strumień aktywności - ekran powiązany z Fragmentem kodu 5 oraz Fragmentem kodu 6.



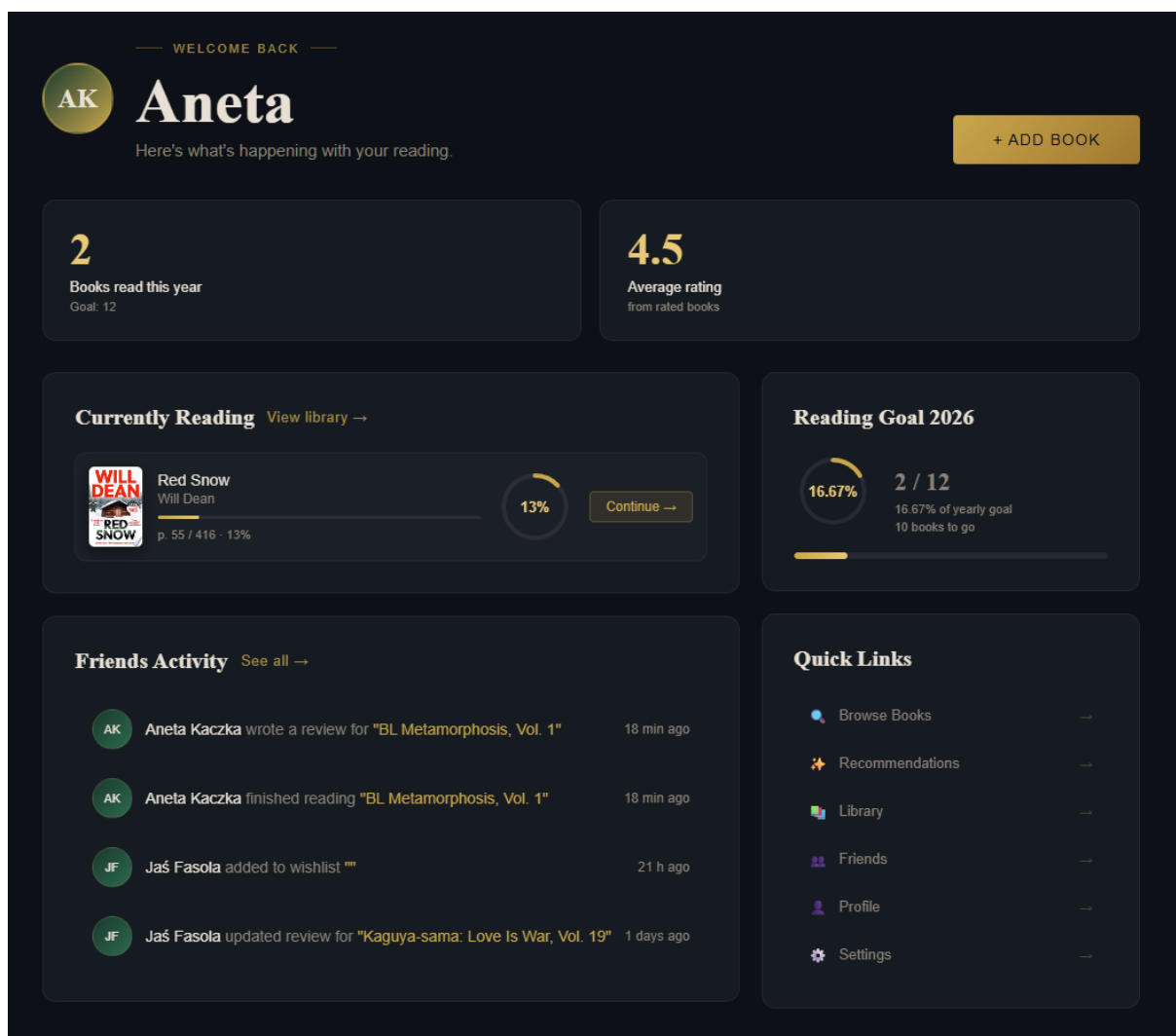
Rysunek 5. Panel czytania ze stoperem, dziennikiem i notatnikiem - ekran powiązany z Fragmentem kodu 7 oraz Fragmentem kodu 8.



Rysunek 6. Sekcja ustawień konta z formularzem bezpiecznego i trwałego usuwania konta - ekran powiązany z Fragmentem kodu 9 oraz Fragmentem kodu 10.



Rysunek 7. Karta szczegółowa książki oraz powiązana seria wydawnicza.



Rysunek 8. Dashboard użytkownika.

8. Testy aplikacji

W celu zapewnienia najwyższej niezawodności oprogramowania, proces weryfikacji poprawności kodu został podzielony na dwa niezależne pakiety testów, uruchamiane osobno dla warstwy serwerowej oraz warstwy klienckiej.

- **Testy warstwy serwerowej:** Do weryfikacji logiki biznesowej API wykorzystano framework xUnit oraz bibliotekę Moq. Celem tych testów było sprawdzenie poprawności działania kontrolerów w pełnej izolacji od fizycznej bazy danych. Poprzez mockowanie interfejsów repozytoriów, zasymulowano zachowania bazy PostgreSQL i sprawdzono, czy kontrolery poprawnie zwracają oczekiwane kody statusów HTTP. Weryfikowano również poprawność kluczowych mechanizmów bezpieczeństwa, takich jak haszowanie haseł BCryptem. Raport z pomyślnego wykonania testów backendu poleceniem *dotnet test* przedstawiono na Rysunku 9.
- **Testy warstwy klienckiej:** Do przetestowania interaktywnych komponentów wizualnych po stronie frontendu wykorzystano biblioteki Vitest oraz React Testing

Library. Celem tych testów było sprawdzenie poprawności renderowania interfejsu oraz zarządzania stanem komponentów Reacta przy interakcjach z użytkownikiem. Testy te weryfikowały również poprawne działanie globalnego dostawcy stanu sesji (AuthProvider) i poprawność walidacji wprowadzanych danych w formularzach w locie. Raport z wykonania testów frontendu przy użyciu skryptu *npm test* przedstawiono na Rysunku 10.

```
C:\Users\akacz\Documents\Github\TrackMyBooks-app\backend\book-service.Tests>dotnet test
Zakończono przywracanie (0,9s)
book-service net10.0 powodzenie (0,6s) → C:\Users\akacz\Documents\Github\TrackMyBooks-app\backend\book-service\bin\Debug\net10.0\book-service.dll
book-service.Tests net10.0 powodzenie (5,4s) → bin\Debug\net10.0\book-service.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.6)
[xUnit.net 00:00:00.13] Discovering: book-service.Tests
[xUnit.net 00:00:00.21] Discovered: book-service.Tests
[xUnit.net 00:00:00.26] Starting: book-service.Tests
Delete account failed: user=User, confirmationValid=False
Delete account failed: user=, confirmationValid=True
Fetching profile for username: ghost
Fetching profile for username: hank
[xUnit.net 00:00:04.08] Finished: book-service.Tests
book-service.Tests test net10.0 powodzenie (5,8s)

Podsumowanie testu: łącznie: 75; niepowodzenie: 0; zakończone powodzeniem: 75; pominięto: 0; czas trwania: 5,8s
Kompiluj powodzenie w 13,6s
```

Rysunek 9. Raport z pomyślnego wykonania testów jednostkowych backendu.

```
This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/wrap-tests-with-act

✓ _components/_tests_/Navbar.test.tsx (2 tests) 257ms
✓ _components/_tests_/FriendsSection.test.tsx (2 tests) 320ms
✓ _components/_tests_/RecommendationsPage.test.tsx (1 test) 334ms
  ✓ should render mood buttons and update content when a mood is selected 330ms
Not implemented: navigation to another Document
✓ _components/_tests_/LoginPage.test.tsx (1 test) 569ms
  ✓ should allow entering credentials and submit the form successfully 565ms

Test Files 4 passed (4)
Tests 6 passed (6)
Start at 10:41:02
Duration 5.42s (transform 571ms, setup 1.75s, import 3.42s, tests 1.48s, environment 12.26s)
```

Rysunek 10. Raport z pomyślnego wykonania testów jednostkowych komponentów frontendu.

9. Wnioski z ćwiczeń

- **Stopień realizacji zakładanych funkcjonalności:** Wszystkie założone w punkcie 3 systemy funkcjonalne zostały pomyślnie zaimplementowane i działają w stopniu w pełni zadowalającym. Użytkownik ma możliwość bezpiecznego rejestrowania konta, logowania się przy użyciu ciasteczek autoryzacyjnych JWT, zarządzania biblioteką na wirtualnych półkach oraz mierzenia postępów czytania za pomocą stopera. Moduł społecznościowy poprawnie agreguje zaproszenia do znajomych i generuje chronologiczną oś czasu aktywności, a silnik rekomendacji trafnie dobiera książki na podstawie nastroju, wykluczając pozycje już posiadane przez czytelnika.
- **Integracja z zewnętrznym API (Hardcover):** Z sukcesem wdrożono komunikację z zewnętrznym interfejsem GraphQL serwisu Hardcover.io za pomocą asynchronicznego klienta HTTP POST. Pobieranie skomplikowanych,

zagnieżdżonych metadanych o książkach w locie pozwoliło na uniknięcie lokalnego składowania milionów rekordów.

- **Optymalizacja interfejsu (useMemo i useRef):** Ciężkie operacje filtrowania i sortowania tysięcy książek po stronie klienckiej zostały zoptymalizowane za pomocą hooka `useMemo`, co całkowicie wyeliminowało opóźnienia interfejsu podczas wpisywania znaków w wyszukiwarce. Z kolei stoper czytania zabezpieczono strukturą `useRef` oraz funkcją czyszczącą w `useEffect`, co zapobiega wyciekowi pamięci w przeglądarce i niepotrzebnym ponownym renderom.
- **Kierunki dalszego rozwoju:** W przyszłości aplikacja mogłaby zostać rozbudowana o wdrożenie technologii WebSockets (np. biblioteki `SignalR` po stronie backendu), co pozwoliłoby na przesyłanie powiadomień o zaproszeniach i aktualizowanie strumienia aktywności znajomych w czasie rzeczywistym bez konieczności odświeżania strony. Dodatkowo warto podpiąć warstwę buforującą (np. bazę Redis lub pamięć podręczną `IMemoryCache` w .NET) dla zewnętrznych zapytań GraphQL, co zabezpieczyłoby system przed limitami żądań (*rate limiting*) narzucanymi przez serwer Hardcover.io.

10. Instrukcja uruchomienia aplikacji i testów

Uruchomienie kompletnego środowiska aplikacyjnego oraz wykonywanie testów jednostkowych odbywa się za pomocą standardowych poleceń konsolowych w odpowiednich katalogach projektu.

a. Uruchomienie aplikacji za pomocą Docker Compose,

Najprostszym i zalecanym sposobem na lokalne uruchomienie systemu jest użycie technologii konteneryzacji Docker, która automatycznie konfiguruje i łączy ze sobą bazę PostgreSQL, backend oraz frontend.

Zbudowanie i pierwsze uruchomienie:

Otwórz terminal w głównym katalogu projektu `TrackMyBooks-app` (tam, gdzie znajduje się plik `docker-compose.yml`) i wykonaj polecenie budujące obrazy: `docker compose up -build`.

Kolejne uruchomienia aplikacji:

Po pomyślnym pierwszym zbudowaniu obrazów, każde kolejne, szybkie uruchomienie kontenerów realizuje się za pomocą polecenia: `docker compose up`.

Adresy i porty usług:

Po poprawnym uruchomieniu kontenerów poszczególne usługi systemu są dostępne pod adresami:

- **Interfejs użytkownika (Frontend):** <http://localhost:3000>
- **Warstwa serwerowa (Backend API):** <http://localhost:5000>
- **Baza danych (PostgreSQL):** localhost:5432

b. Uruchamianie testów jednostkowych

Testy jednostkowe obu warstw aplikacji są uruchamiane lokalnie z poziomu wiersza poleceń w dedykowanych folderach projektów testowych.

Uruchamianie testów backendowych (C# / xUnit):

Otwórz terminal i przejdź do katalogu projektu testowego backendu: `cd backend\book-service.Tests`. Następnie uruchom pakiet testów jednostkowych C# za pomocą polecenia: `dotnet test`.

Uruchamianie testów frontendowych (React / Vitest):

Otwórz terminal i przejdź do katalogu projektu frontendowego: `cd frontend\track-my-books`. Następnie uruchom pakiet testów komponentów klienckich za pomocą polecenia: `npm test`.